

Projekt #2 – drzewa prefiksowe

Projekt polega na zaimplementowaniu drzewa prefiksowego, czyli struktury danych służącej do przechowywania słów oraz szybkiego wyszukiwania słów o danym prefiksie. Celem projektu jest sprawdzenie swoich kompetencji w implementacji struktur drzewiastych i zapoznanie ze strukturą *drzewa prefiksowego*.

Ogólne zasady

- Projekt należy wykonać w języku Python.
- Wynikiem projektu powinny być dwa pliki:
 - `trie.py` – implementacja klas `TrieNode` i `Trie`
 - `tests.py` – skrypt prezentujący działanie zaimplementowanych klas
- Kod powinien być czytelny i uporządkowany. Zmienne nazywamy w czytelny sposób w języku angielskim.
- Zakazane jest korzystanie z narzędzi AI do zaimplementowania kodu (natomiast można oczywiście się skonsultować).
- Pliki `.py` umieszczamy w folderze `projekt2_<imię>_<nazwisko>.zip` i przesyłamy mailem na adres `jakszy9@amu.edu.pl` do
- Za wykonanie projektu można otrzymać do 25 punktów (zgodnie z zasadami zaliczenia).

Drzewa prefiksowe

Drzewo prefiksowe (trie) to struktura danych postaci drzewa, gdzie każdy węzeł przechowuje jeden symbol (jedną literę). Ścieżki od korzenia do liści drzewa kodują ciągi znaków (słowa) przechowywane w strukturze.

Drzewo prefiksowe dla słów kot, kota, komar, pies możemy wyobrazić sobie

jak po prawej. Ścieżki

```
(root) --> (k) --> (o) --> (m) --> (a) --> r
(root) --> (k) --> (o) --> (t) --> (a)
```

współdzielą ze sobą początkowe litery:

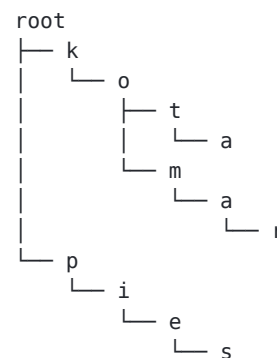
```
(root) --> (k) --> (o),
```

ale `ko` nie jest rozpoznawalnym przez strukturę słowem (nie ma `go` w słowniku). Ścieżka

```
(root) --> (k) --> (o) --> (t)
```

jest podścieżką `kota`, ale same w sobie również reprezentuje słowo.

Stąd w drzewach prefiksowych każdy węzeł musi zostać wzbogacony o informację, czy jest on miejscem końcowym dla pewnej ścieżki.



Wymagania

Pełne rozwiązanie powinno zawierać implementację obu opisanych poniżej klas (plik `trie.py`) oraz osobny plik importujący obie klasy i prezentujący ich działanie, w tym działanie ich metod, na przykładowej liście słów (plik `tests.py`).

- **klasa `TrieNode`**

Węzeł drzewa prefiksowego. Każdy węzeł przechowuje jeden znak i wskazuje na węzły potomne.

Atrybuty:

- `char: str` – znak przypisany do węzła
- `children: dict[str, TrieNode]` – słownik węzłów potomnych (klucz: znak, wartość: węzeł)
- `is_end: bool` – flaga oznaczająca, czy węzeł kończy pewne słowo w strukturze

Konstruktor `__init__(self, char: str)` inicjalizuje węzeł.

- **klasa Trie**

Drzewo prefiksowe. Korzeń nie przechowuje żadnego znaku – jest jedynie punktem startowym.

Konstruktor `__init__(self)` tworzy korzeń drzewa (`root`) jako węzeł `TrieNode` z pustym znakiem.

Metody:

- `insert(self, word: str) -> None`
Wstawia słowo `word` do drzewa.
- `search(self, word: str) -> bool`
Zwraca `True`, jeśli słowo `word` znajduje się w drzewie. W przeciwnym razie zwraca `False`.
- `starts_with(self, prefix: str) -> bool`
Zwraca `True`, jeśli w drzewie istnieje choć jedno słowo zaczynające się od `prefix`. Nie wymaga, żeby `prefix` sam w sobie był słowem.
- `get_words_with_prefix(self, prefix: str) -> list[str]`
Zwraca listę wszystkich słów w drzewie zaczynających się od `prefix`. Jeśli żadne słowo nie pasuje, zwraca pustą listę.